

Ejercicio de generación de Árboles de Sintaxis Abstracta binarios (ASTb).

El uso del código diferido es útil para realizar traducciones sobre la marcha durante la fase del Análisis Sintáctico.

Sin embargo presenta algunas desventajas. Una de ellas es la gestión de memoria dinámica. Por un lado no estamos liberando memoria (esto para simplificar el código). Pero en cualquier caso, cada nivel en la gramática en la que se concatenan varias cadenas previas puede suponer duplicar la memoria reservada, es decir, el consumo de memoria puede crecer de forma exponencial. Al finalizar la traducción de un programa obtendríamos una única y enorme cadena con toda la traducción.

Se propone aquí una forma alternativa para almacenar la información traducida en base a estructuras de datos en forma de árboles. Es la que se usa en muchos compiladores y traductores, los así llamados AST: Árbol de Sintaxis Abstracta.

Este tipo de estructura presenta una ventaja adicional: Una vez construida es más fácil aplicar comprobaciones de tipo de Análisis Semántico mediante funciones de recorrido del árbol.

En el presente ejercicio nos ceñimos a un tipo de árbol binario adecuado para expresiones aritméticas. Y además se plantea usarlos para traducir notación infija en notación prefija. Esto restringe su uso al caso de expresiones binarias. Una simplificación adicional es que se aprovecha el proceso de construir el árbol para insertar la información de traducción.

El código bison comienza con la primera sección en la que se definen las funciones y otros recursos habituales.

Debéis integrar en vuestro código las novedades.

```

%{                                     // SECTION 1
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

extern int yylex () ;
extern int yyerror () ;

char temp [2048] ;

#define FF fflush(stdout);

char *my_malloc (int nbytes) {
    char *p ;
    static long int nb = 0;
    static int nv = 0 ;

    p = malloc (nbytes) ;
    if (p == NULL) {
        fprintf (stderr, "No memory available for %d bytes\n", nbytes) ;
        fprintf (stderr, "%ld bytes allocated in %d calls\n", nb, nv) ;
        exit (0) ;
    }
    nb += (long) nbytes ;
    nv++ ;

    return p ;
}

char *gen_code (char *nombre)        // this is roughly equivalent to strdup()
{
    char *p ;
    int l ;

    l = strlen (nombre)+1 ;
    p = (char *) my_malloc (l) ;
    strcpy (p, nombre) ;

    return p ;
}

char *char_to_string (int c) {
    sprintf(temp, "%c", c);           // convert character to string
    return gen_code (temp) ;
}

char *int_to_string (int n) {
    sprintf(temp, "%d", n);           // convert int to string
    return strdup (temp) ;// strdup does the same as gen_code
}

```

Definimos la estructura del nodo para construir el árbol binario para almacenar la información del AST. Serán necesarios:

- Dos campos para enlazar con el hijo izquierdo y el derecho.
- Otro campo para almacenar el *token*, o el operador o su traducción a Lisp (caso de $= \Rightarrow \text{setq}$) en formato string.
- Otro campo para almacenar si se trata de un nodo binario, de un nodo con un único nodo hijo (menos unario), o los nodos hoja que almacenan la información de los token (Variables y Números).

// Abstract Syntax Tree (AST) Node Structure

```
typedef struct ASTnode t_node ;
```

```
struct ASTnode {  
    char *op ;           // simplified node information in form as a string  
    int type ;           // leaf, unary or binary nodes (0/1/2)  
    t_node *left ;  
    t_node *right ;  
} ;
```

Definimos la función primitiva para crear un nodo. Los argumentos son:

- El operador (o token) ya traducido en formato string.
- El tipo de nodo (valores 2/1/0) para diferenciar si se trata de un nodo binario, uno unario con un único nodo hijo (menos unario), o los nodos hoja que almacenan la información de los *token* (Variables y Números).
- Dos punteros a *t_node* para pasar los enlaces a los nodos hijo izquierdo y derecho.

// Function to create a new AST node

```
t_node * createASTNode (char *op, int type, t_node *left, t_node *right) {  
    t_node *node ;  
  
    node = (t_node*) malloc (sizeof (t_node)) ;  
    node->op = strdup (op) ;           // duplicates string in memory  
    node->type = type ;                 // binary, unary or leaf node  
    node->left = left ;  
    node->right = right ;  
    return node ;  
}
```

Definimos otra función primitiva para liberar un nodo.

// Function to free AST

```
void freeAST (t_node *node) {  
    if (node != NULL) {  
        freeAST (node->left) ;  
        freeAST (node->right) ;  
        free (node->op) ;  
        free (node) ;  
    }  
}
```

Definimos también una función específica para recorrer el AST en preorden imprimiendo el contenido en notación prefija de LISP.

Los paréntesis se imprimen únicamente en caso de nodos intermedios de 1 o 2 hijos.

```
// Function to print AST in prefix notation

void printAST2Prefix (t_node *node) {
    if (node == NULL) {
        return ;
    } else if (node->type == 0) {
        printf ("%s ", node->op) ;    // Leaf node
    } else if (node->type == 1) {
        printf ("( %s ", node->op) ;    // Unary node
        printAST2Prefix (node->left) ;
        printf (") ") ;
    } else {
        printf ("( %s ", node->op) ;    // Binary node
        printAST2Prefix (node->left) ;
        printAST2Prefix (node->right) ;
        printf (") ") ;
    }
}
```

Las siguientes definiciones son las habituales de las prácticas con bison.

Se incluye en la estructura de pila de bison el nuevo atributo .node, que servirá para transmitir el puntero a los subárboles AST creados.

```
typedef struct s_attr {
    int value ;                // .valor in spanish version
    int index ;
    char *code ;              // .codigo in spanish version
    t_node *node ;            // for AST
} t_attr ;
```

```
#define YYSTYPE t_attr
```

```
%}
```

```
// SECTION 2
```

```
%token  NUMBER
```

```
%token  VARIABLE
```

```
%right  '='                // operator with the least precedence
```

```
%left  '+' '-'            //
```

```
%left  '*' '/'            //
```

```
%left  UNARY_SIGN // operator with the highest precedence
```

```
%%
```

// SECTION 3: Grammar - Semantic Section

```
axiom:      expression '\n'      { }
           r_expr                { }
       | VARIABLE '=' expression '\n' { }
           r_expr                { }
       ;

r_expr:     /* lambda */        { }
       | axiom                  { }
       ;

expression: term                 { }
       | expression '+' expression { }
       | expression '-' expression { }
       | expression '*' expression { }
       | expression '/' expression { }
       ;

term:       operand              { }
       | '+' operand %prec UNARY_SIGN { }
       | '-' operand %prec UNARY_SIGN { }
       ;

operand:    VARIABLE             { }
       | NUMBER                  { }
       | '(' expression ')'      { }
       ;

%%
```

```

/* SECTION 4 C Code */

int n_line = 1 ;

int yyerror (char *message)
{
    fprintf (stderr, "%s in line %d\n", message, n_line) ;
}

int yylex ()
{
    unsigned char c ;

    do {
        c = getchar () ;
    } while (c == ' ') ;

    if (c == '.' || (c >= '0' && c <= '9')) {
        ungetc (c, stdin) ;
        scanf ("%d", &yylval.value) ;
        return NUMBER ;
    }

    if (c >= 'A' && c <= 'Z') {
        yyval.index = c - 'A' ;
        return VARIABLE ;
    }

    if (c >= 'a' && c <= 'z') {
        yyval.index = c - 'a' ;
        return VARIABLE ;
    }

    if (c == '\n')
        n_line++ ;
    return c ;
}

int main ()
{
    yyparse () ;
}

```

Es necesario insertar en las acciones semánticas las llamadas a la función primitiva de creación de un nodo de forma que se puedan ir construyendo fragmentos de AST que se van enlazando al ascender por el árbol de derivación.

En el axioma debe incluirse una llamada a la impresión del árbol en formato prefijo.

Ejemplos de distintos casos de uso de la función primitiva de creación de nodos:

Para el token Variable sería un nodo hoja:

```
$$ .node = createASTNode (char_to_string ($X.index+'A'), 0, NULL, NULL) ;
```

Para el signo - unario:

```
$$ .node = createASTNode ("-", 1, $X.node, NULL) ;
```

Considerad la posibilidad de crear en éste caso un nodo binario correspondiente a:

```
(- 0 <expr>)
```

Para un operador binario (por ejemplo, la suma):

```
$$ .node = createASTNode ("+", 2, $X.node, $Y.node) ;
```

Donde \$X, \$Y representan los No Terminales y Token involucrados en la producción que se enlazan como nodos hijo izquierdo y derecho.